



Search ...



Product Categories

Learn

Videos

Deals & Discounts

Questions?



About Us

Product Categories



Learn



Login / Register

About Us

Questions?

Videos

FUNDAMENTALS

ESP8266 Pinout Reference: How To Use ESP8266 GPIO Pins



Written by Ali Akbar Hosseini



Introduction

ESP chips and NodeMCU boards have a crucial role in some areas, such as smart homes and electronics, because they have good features and a reasonable price. These boards are quite useful for various projects. It is also very easy to work with them. The Arduino IDE software may even be used to program NodeMCU boards.

Words like “ESP” and “NodeMCU” might be a bit confusing for beginners who have just stepped into this field as to whether they mean the same thing. Therefore, we will first describe “ESP8266” and “NodeMCU,” their various models, and their differences to help you make the best choice based on your project.

The helpful specifications of the pins, which include practically all the relevant information, will then be described in a few images. If you are not so familiar with these boards, you can print the photos and put them on your desk to find what you need at first glance.

We'll next go into more detail about different parts of the NodeMCU ESP8266 board, how it works, and how to write code in Arduino IDE. At the end of this tutorial, not only will we get to know the ESP8266 pins but we also learn the principles of how to use them.

What You Will Learn

- Difference between ESP8266 and NodeMCU, as well as their various versions
- Programming of NodeMCU boards using Arduino IDE software
- ESP8266 Board Pinout
- Which GPIO pins can be used on ESP8266 Board
- How to use different parts of the board, communication protocols, and ...
- Which pins to be careful with



Is ESP8266 the same as NodeMCU? Different Versions of NodeMCU ESP8266

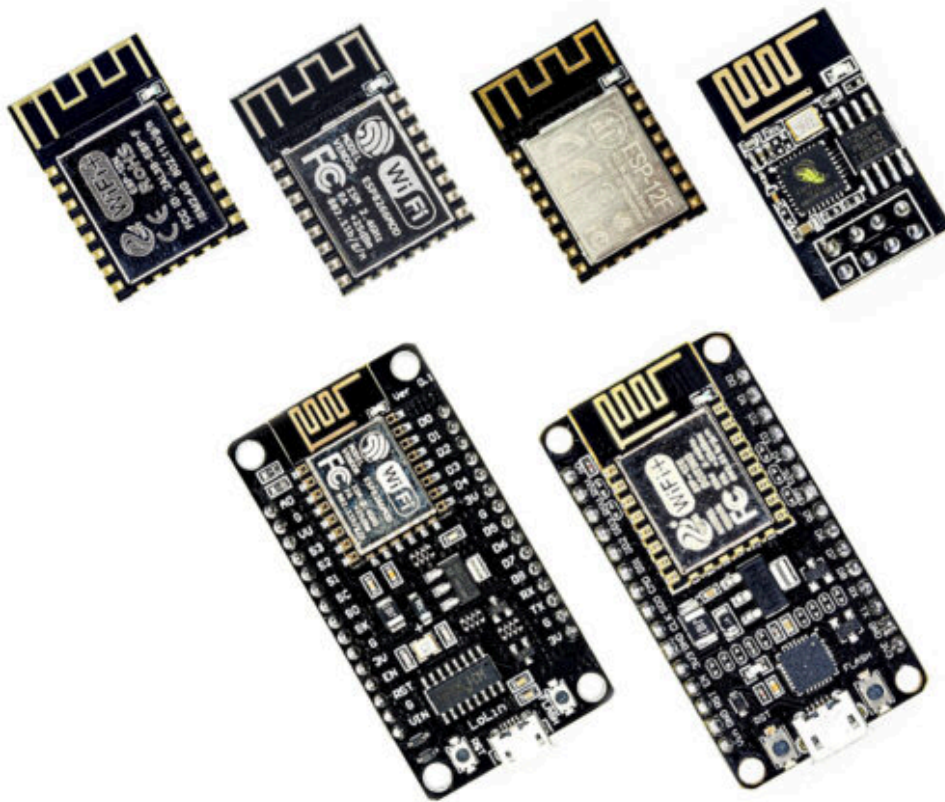
ESP8266 is a WiFi microcontroller from the ESP family that requires external flash memory and an antenna.

For this microcontroller, which is sometimes referred to as an “ESP Module” or simply “ESP,” various modules and boards have been made.

What is a Development Board?

Development boards based on a component (for example, development boards based on ESP) are the boards that are designed to work with a certain IC or component. They provide more features and make it easier to work with such components.

NodeMCU is, in fact, an IoT platform that includes the firmware which runs on the ESP8266. However, ESP8266-based development boards with NodeMCU firmware are also called NodeMCU boards. (Put more simply: NodeMCU is a development board whose central part, and indeed the only core of the board, is an ESP8266 (recently an ESP32) microcontroller.)



SOME MODELS
OF ESP8266
WIFI MODULES

SOME MODELS
OF ESP8266
DEVELOPMENT
BOARDS

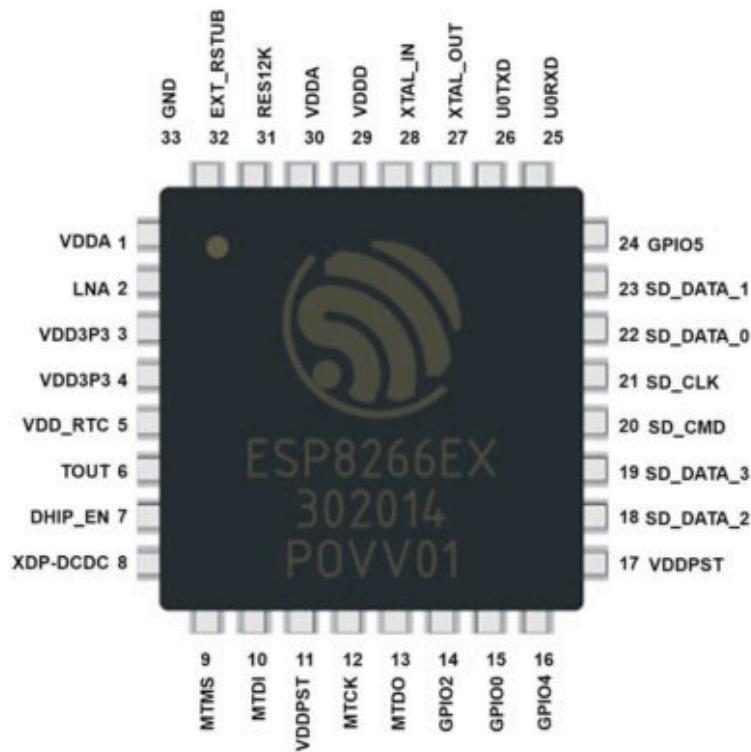


So, while ESP8266 and NodeMCU are not the same, ESP8266 may be wrongly referred to as a NodeMCU development board that features the ESP8266 microcontroller.

ESP8266 Pinout

It is crucial to master ESP8266 pinout so you can use every GPIO pin in the right way. Here is the pinout for different models of ESP8266.

Datasheet and Pinout For ESP8266 IC

**ESP8266 IC Datasheet**

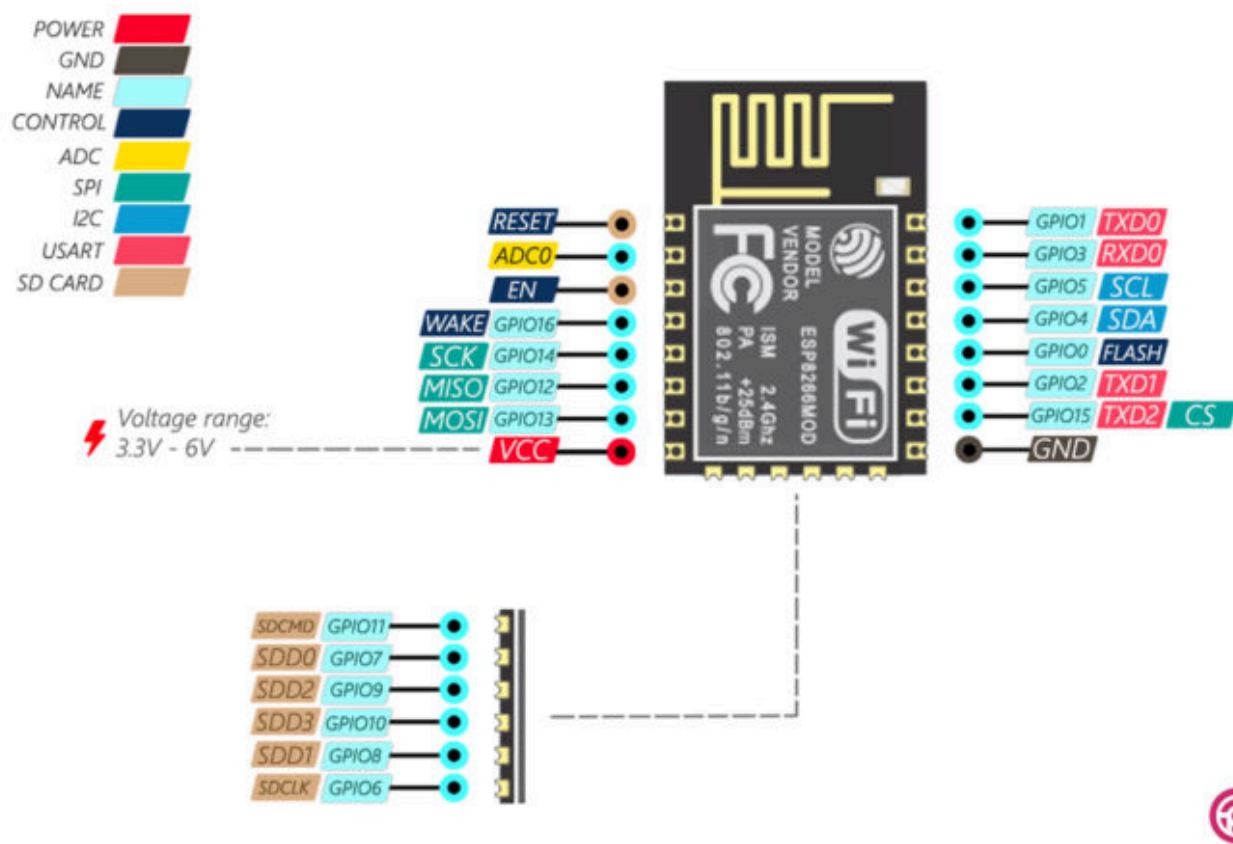
1.26 MB



769 downloads

[Download](#)

ESP8266 Pinout And Datasheet (ESP8266-12F Module)



Datasheet of ESP8266-12F WiFi Module

1.15 MB 1199

Download

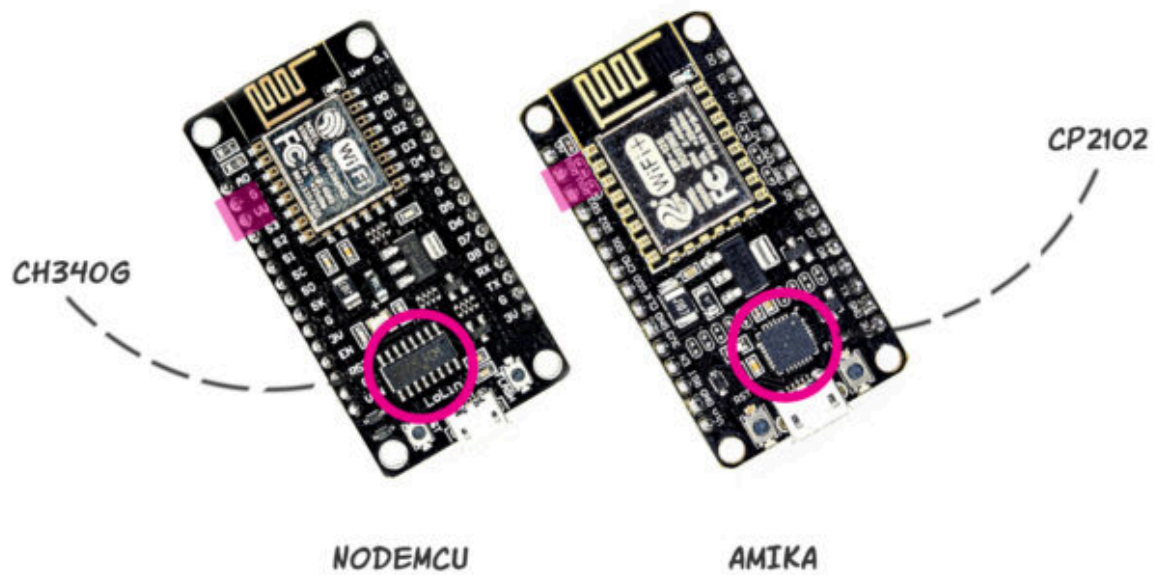
downloads

...

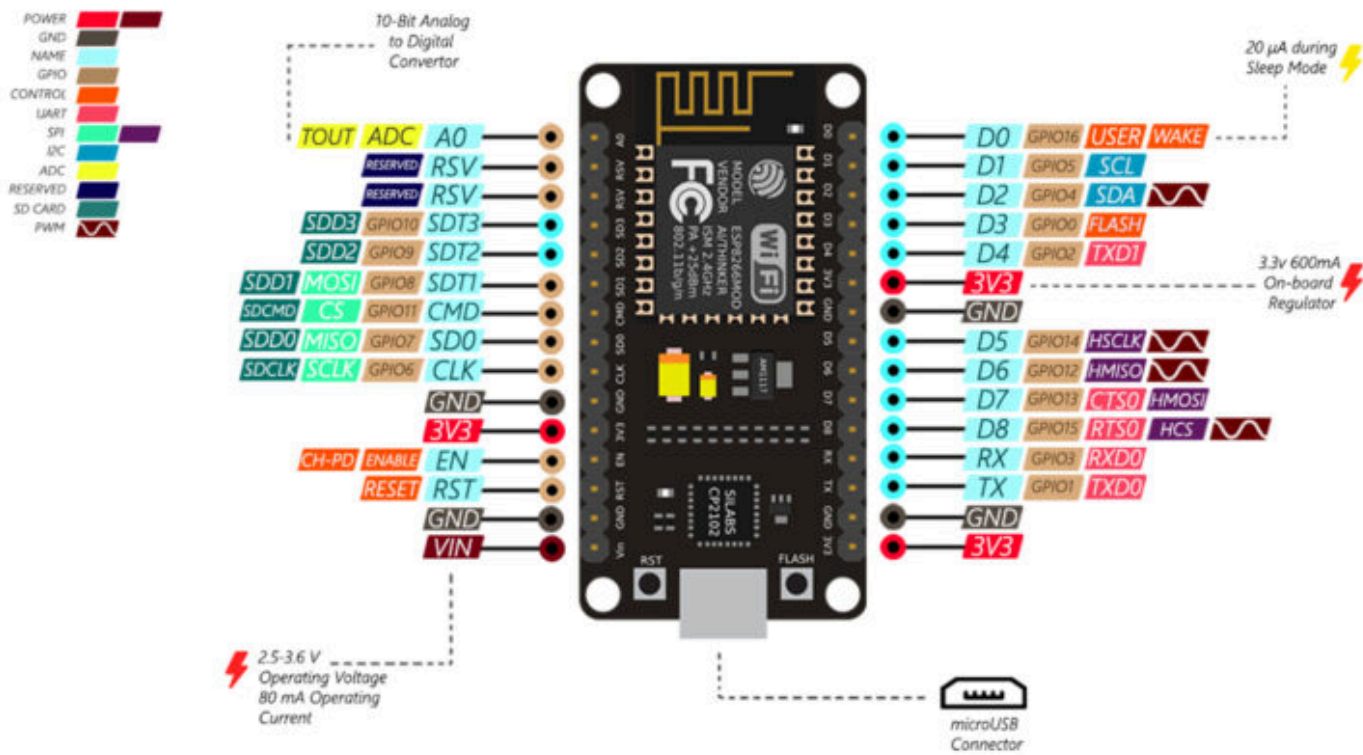
This tutorial will describe the pins of the NodeMCU ESP8266 development board, which operates on the same principles as the ESP8266 module.

NodeMCU ESP8266 Pinout (Amica/DoIT, LoLin Brands)

Amica/DoIT models are the second generation of NodeMCU development boards, whereas the LoLin model is the third generation.



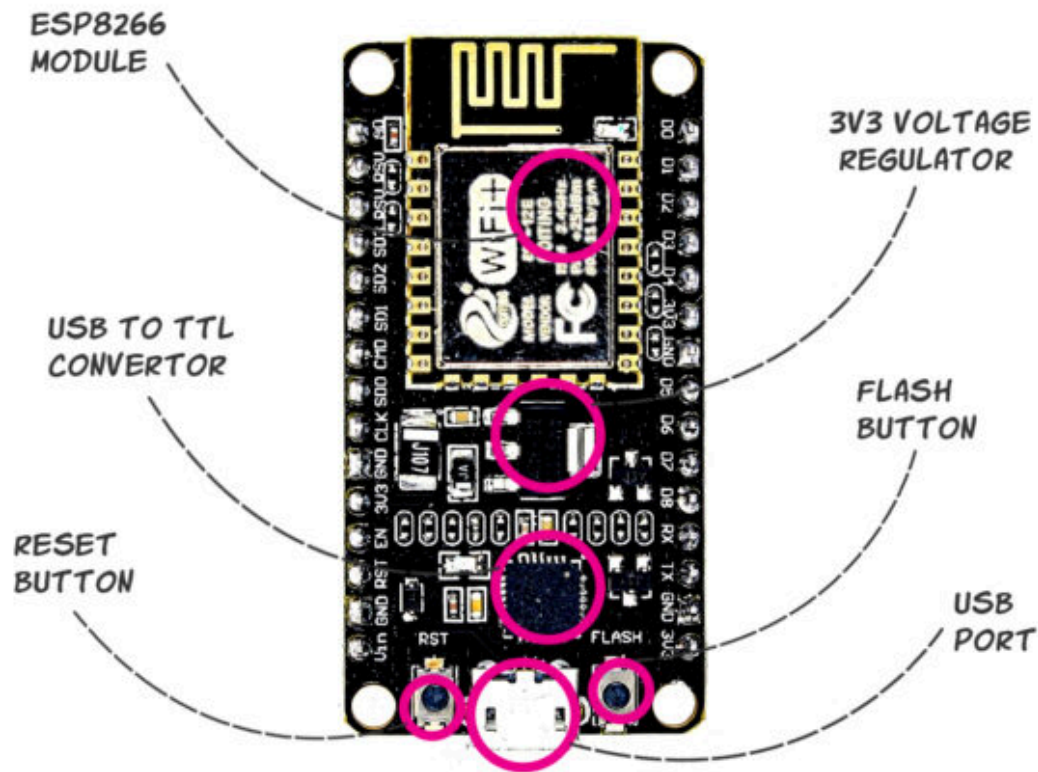
Today, the most popular versions in the market are NodeMCU LoLin and NodeMCU Amica models, which differ in dimensions and pins #2 and 3. But the rest of their pins' specifications are the same.



NodeMCU Specifications

NodeMCU Technical Specifications		
	Official NodeMCU <u>Amica</u>	<u>LoLin</u> NodeMCU
Microcontroller	ESP-8266 32-bit	ESP-8266 32-bit
NodeMCU Model	<u>Amica</u>	Clone <u>LoLin</u>
NodeMCU Size	49mm x 26mm	58mm x 32mm
Pin Spacing	0.9" (22.86mm)	1.1" (27.94mm)
Clock Speed	80 MHz	80 MHz
USB to Serial	CP2102	CH340G
USB Connector	Micro USB	Micro USB
Operating Voltage	3.3V	3.3V
Input Voltage	4.5V-10V	4.5V-10V
Flash Memory/SRAM	4 MB / 64 KB	4 MB / 64 KB
Digital I/O Pins	11	11
Analog In Pins	1	1
ADC Range	0-3.3V	0-3.3V
UART/SPI/I2C	1 / 1 / 1	1 / 1 / 1
<u>WiFi</u> Built-In	802.11 b/g/n	802.11 b/g/n
Temperature Range	-40C - 125C	-40C - 125C

Main Components on NodeMCU ESP8266 Board



NodeMCU ESP8266 Pinout And Specifications; All in One Look

In the following image you can see all important info of an ESP8266 board. If you use ESP8266 boards regularly, you can print it and stick it on your desk.

10 BIT ANALOG TO DIGITAL CONVERTOR, INPUT RANGE 0-3.3 VOLT		ANALOG INPUT	
RESERVED, NO FOR USE		GND AT LOLIN VERSION	
RESERVED, NO FOR USE		5V AT LOLIN VERSION	
THESE ARE SPI PINS(SCK, CMD, SDO, CLK) WHICH ARE EXCLUSIVELY USED FOR QUAD-SPI COMMUNICATION WITH FLASH MEMORY ON ESP-32E, HENCE, THEY CANT BE USED FOR SPI APPLICATIONS. IT CONSISTS OF QUAD I/O (4-BIT DATA BUS) I.E. FOUR (SDIO_ DATA0-SDIO_ DATA3) BIDIRECTIONAL I/O AND O/P DATA SIGNALS WITH SYNCHRONIZING CLOCK (SDIO_ CLK) AND CHIP SELECT PIN (SDIO_ CMD)		SDIO _ DATA3	
		SDIO _ DATA2	
		SDIO _ DATA1	
		SDIO_ CMD(CS)	
		SDIO _ DATA0	
GROUND		SDIO _ SCLK	
3.3V, REGULATED WITH ONBOARD REGULATOR, 600MA MAX			
THE ESP8266 CHIP IS ENABLED WHEN EN PIN IS PULLED HIGH			
RESET THE BOARD IF PULL LOW, CONNECTED TO THE RESET BUTTON			
GROUND			
INPUT VOLTAGE 4.5 - 10V			

ESP8266	
GPIO 16/D0	NO INTERRUPT
GPIO 5/D1	INPUT OK
GPIO 4/D2	INPUT OK
GPIO 0/D3	PULLED UP
GPIO 2/D4	PULLED UP
3V3	3.3V, REGULATED WITH ONBOARD REGULATOR, 600MA MAX
GND	GROUND
GPIO 14/D5	INPUT OK
GPIO 12/D6	INPUT OK
GPIO 13/D7	INPUT OK
GPIO 15/D8	PULLED TO GND
GPIO 3/EX	NOT RECOMMENDED
GPIO 1/TX	X
GND	GROUND
3V3	3.3V, REGULATED WITH ONBOARD REGULATOR, 600MA MAX

INPUT STATE		OUTPUT STATE		OTHER FUNCTION/DESCRIPTION	
NO INTERRUPT	NO PWM OR DC	HIGH AT BOOT	USE TO WAKE UP FROM DEEP SLEEP		
INPUT OK	OUTPUT OK (MOST RECOMMENDED)	SCL (I2C)			PWM
INPUT OK	OUTPUT OK (MOST RECOMMENDED)	SDA (I2C)			PWM
PULLED UP	OUTPUT OK	CONNECTED TO FLASH BUTTON	BOOT FAILS IF PULLED LOW		PWM
PULLED UP	OUTPUT OK	HIGH AT BOOT	TXD1	BOOT FAILS IF PULLED LOW	PWM
3.3V, REGULATED WITH ONBOARD REGULATOR, 600MA MAX					
GROUND					
INPUT OK	OUTPUT OK	SCLK (SPI)			PWM
INPUT OK	OUTPUT OK	MISO (SPI)			PWM
INPUT OK	OUTPUT OK	MOSI (SPI)	TXD2		PWM
PULLED TO GND	NOT RECOMMENDED	CS (SPI)	TXD2	BOOT FAILS IF PULLED HIGH	PWM
NOT RECOMMENDED		HIGH AT BOOT	TXD0		
X	NOT RECOMMENDED	HIGH AT BOOT	TXD0	BOOT FAILS IF PULLED LOW	
GROUND					
3.3V, REGULATED WITH ONBOARD REGULATOR, 600MA MAX					



ESP8266 Pinout: Suitable Pins for Digital I/O

First Priority

The most suitable pins for digital I/O are D1 (GPIO5) and D2 (GPIO4).

Second Priority

D5 (GPIO14), D4 (GPIO12), and D7 (GPIO13) pins can be used as digital I/O. The reason is that these pins are for SPI communications, and if we use them as digital I/O, we won't need to to use the NodeMCU Board's SPI Protocol (which is used for setup and communication by many devices, including some monitors). The SPI limitation won't be an issue if the circuit doesn't have such a component, of course.

Third Priority

D3 (GPIO0) and D4 (GPIO2) can also function as digital outputs. Remember that the boot process uses these two pins. In other words, the board won't boot if these pins are pulled HIGH or LOW by external components such as a relay, optocoupler, transistor, or any driver model. Setting these pins to LOW will result in the circuit being unable to boot. These two pins are pulled UP internally. And D3 (GPIO0) is connected to the board's FLASH Button.

Other Pins Which Are Not Recommended to Use

We can set the TX pin (GPIO1) as an output, although it is not recommended. This pin serves as the data transmission line through the serial port, which is also used to communicate with the computer. Plus, it is HIGH during boot, and changing it to LOW will stop the boot process.

We can use the RX pin (GPIO3) as an input. This pin is the data receiving line through the serial port, which is also used to communicate with the computer. And it's also HIGH at boot time.

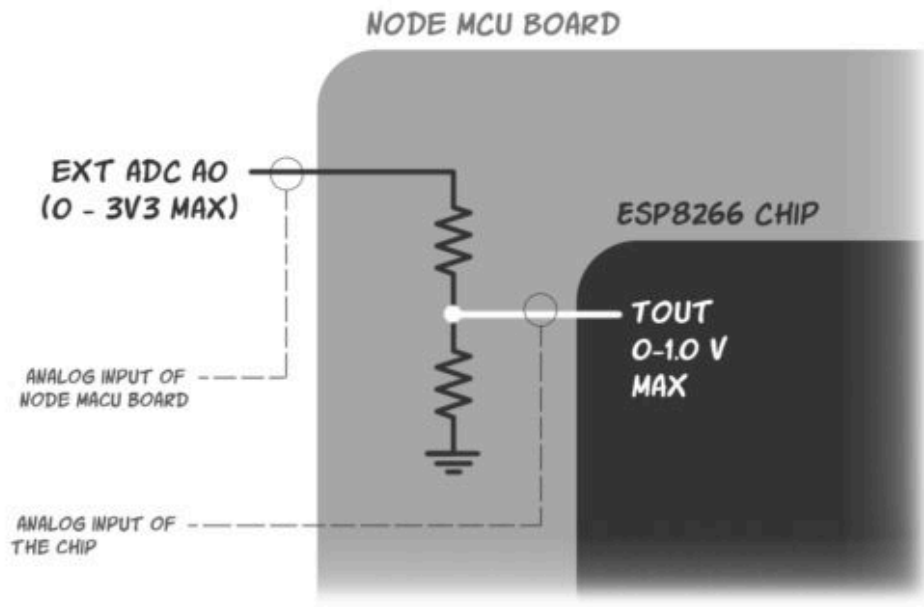
The D0 pin (GPIO16) has no interrupt as a routine input and does not support PWM or I2C as an output. Likewise, it is HIGH at boot.

We can set the D8 pin (GPIO15) as an output. The board won't be able to boot if it becomes HIGH during boot.

ESP8266 Pinout: ADC Pins with Arduino IDE

ADC (Analog-to-Digital Converter) is a unit that converts analog values to digital values. For example, in order to use the output of a sensor with an analog voltage value in a microcontroller, the sensor's output must be read using an analog input and converted into a digital value.

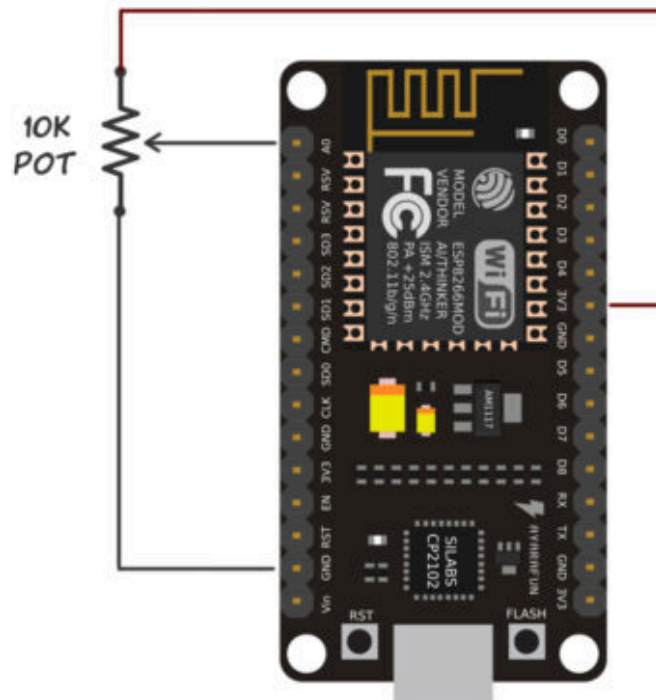
The 10-bit ADC channel on the ESP8266 chip or module has a voltage range of 0-1V. In the NodeMCU Board, this range has been expanded to 0-3.3V. This increase is achieved by just a simple resistive division. We can increase the range as seen in the figure below.



The only analog input on the NodeMCU ESP8266 Board is the A0 pin (ADC0).

In the following example, the voltage of the potentiometer's middle pin is read and displayed in the serial window.

Schematic and Code



```

1
2 void setup() {
3   Serial.begin(9600);
4 }
5
6 void loop() {
7   Serial.print("ADC Value: ");Serial.println(analogRead(A0));
8   delay(300);
9 }

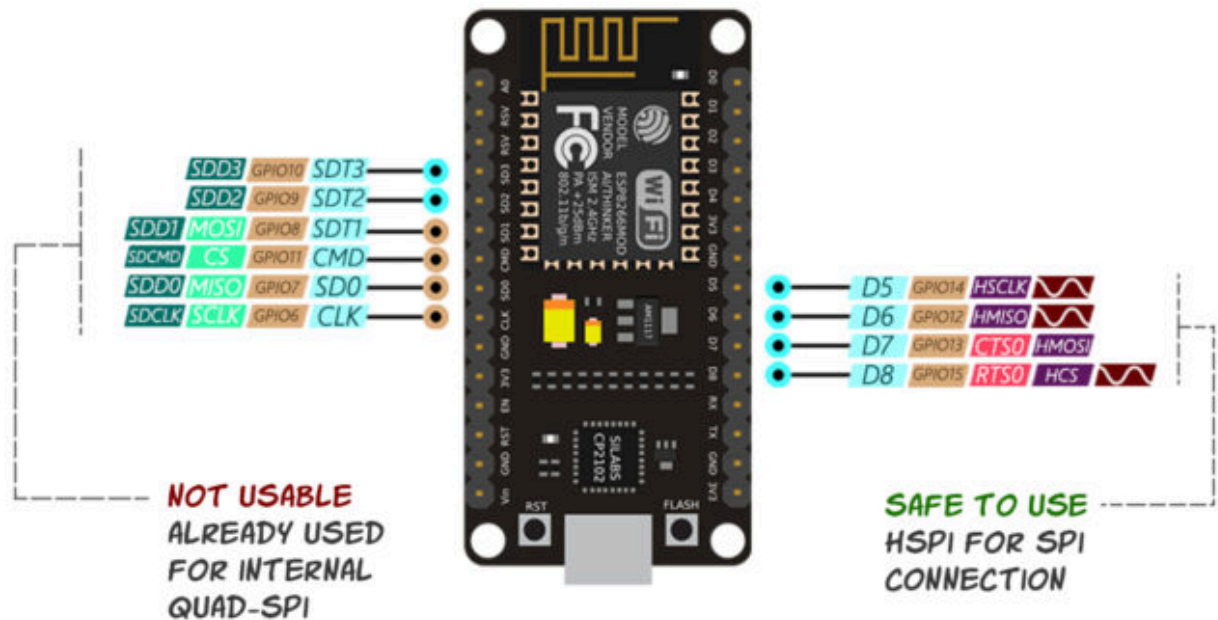
```

ESP8266 Pinout: SPI Pins

On the left side of the image, there are six pins (Data Lines (SD3, SD2, SD1, SD0), Clock Line (SDIO_CLK), and the Chip Select (SDIO_CMD)) which are used for the internal Quad-SPI communication with the flash memory. These pins have practically no external uses and should be avoided.

In addition, the NODEMCU ESP8266 Board has a hardware SPI and provides four pins for SPI

communications (D5, D6, D7, D8). Any device that supports The SPI Protocol can be connected to these four pins.

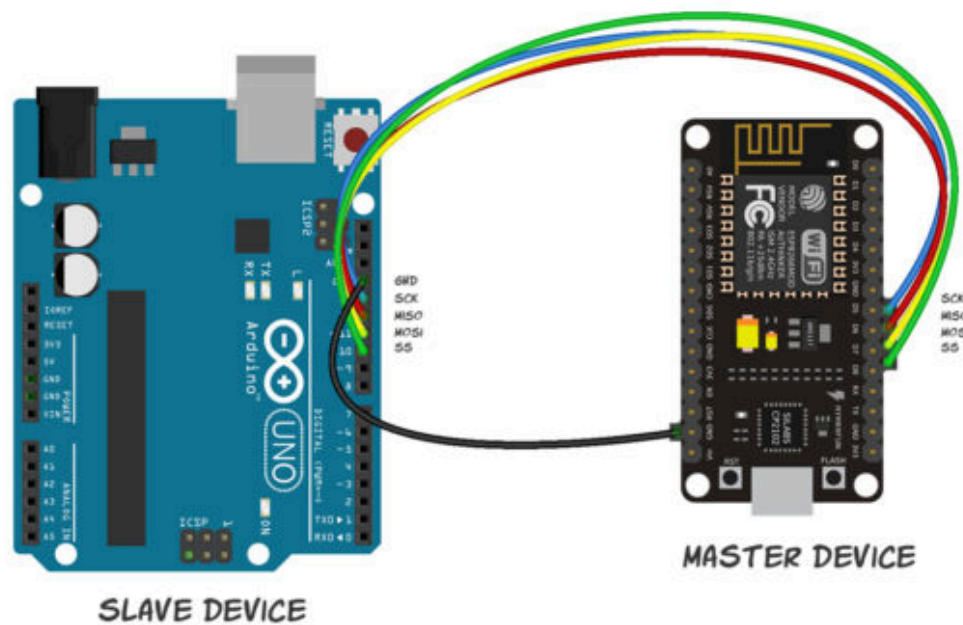


- D5 (GPIO14) HCLK SPI Clock pin
- D6 (GPIO12) HMISO MISO pin (Master In Slave Out)
- D7 (GPIO13) HMOSI MOSI pin (Master Out Slave In)
- D8 (GPIO15) HCS CS pin (Chip Select); the Master can select the desired Slave by communicating with this pin.

Example: SPI Communication between NodeMCU Board and Arduino UNO

In the example below, Arduino is configured as a Slave and NodeMCU as a Master. The Slave receives the message "HELLO UNO" from the Master and displays it on the serial port.

Schematic



Code for NodeMCU as a Master

```

1
2  #include<SPI.h>
3  char buff[]="Hello UNO\n";
4  void setup() {
5      SPI.begin(); /* begin SPI */
6  }
7
8  void loop() {
9      for(int i=0; i<sizeof buff; i++){ /* transfer buff data per second */
10         SPI.transfer(buff[i]); // transfer byte i from buffer buff on SPI line
11         delay(1000);
12     }
13 }

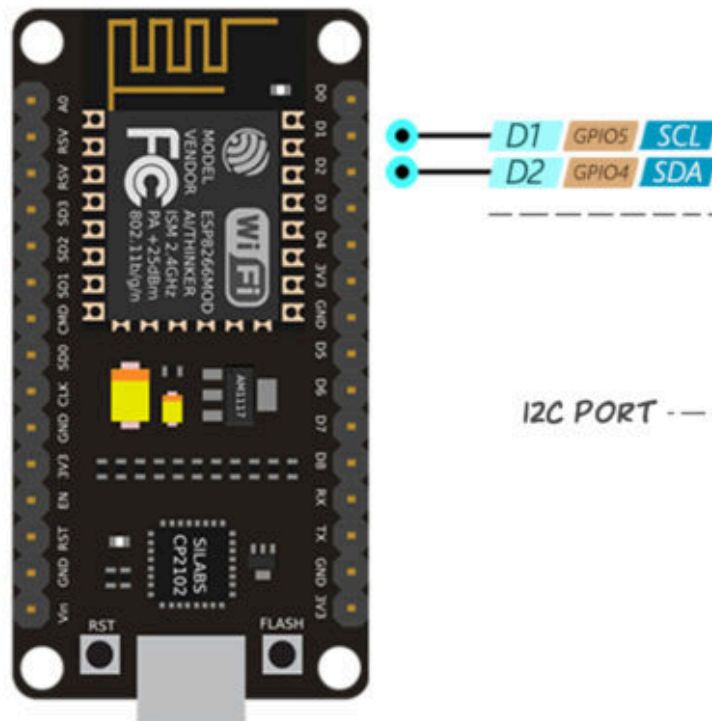
```

Code for Arduino UNO as a Slave

```
1
2  #include <SPI.h>
3
4  char buff [100];
5  volatile byte index;
6  volatile bool receivedone; /* use reception complete flag */
7
8  void setup (void)
9  {
10   Serial.begin (9600);
11   SPCR |= bit(SPE); /* Enable SPI */
12   pinMode(MISO, OUTPUT); /* Make MISO pin as OUTPUT */// <<---*** in slave device MISO
13   index = 0;
14   receivedone = false;
15   SPI.attachInterrupt(); /* Attach SPI interrupt */// activate spi interrupt
16 }
17
18 void loop (void)
19 {
20   if (receivedone) /* Check and print received buffer if any */
```

ESP8266 Pinout: I2C Pins

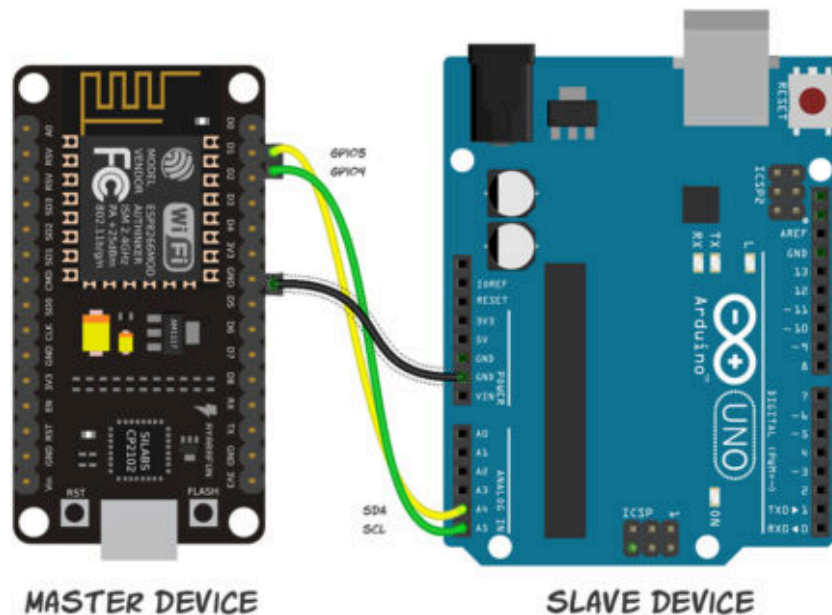
D1 (GPIO5) and D2 (GPIO4) pins are suitable for I2C communications.



I2C Communications between NodeMCU and Arduino UNO using Arduino IDE Software

In the example below, the Master NODEMCU sends a “hello” string message to the Slave Arduino, and vice versa. The serial window can display the messages sent to the computer’s serial port.

I2C Communication Schematic



Code for NodeMCU ESP8266 Board

```
#include <Wire.h> // library for i2c communication

void setup() {
  Serial.begin(9600); /* begin serial for debug */
  Wire.begin(D1, D2); /* join i2c bus with SDA=D1 and SCL=D2 of NODEMCU */
}

void loop() {
  Wire.beginTransmission(8); /* begin with device address 8 */
  Wire.write("Hello Arduino"); /* sends hello string */
  Wire.endTransmission(); /* stop transmitting */

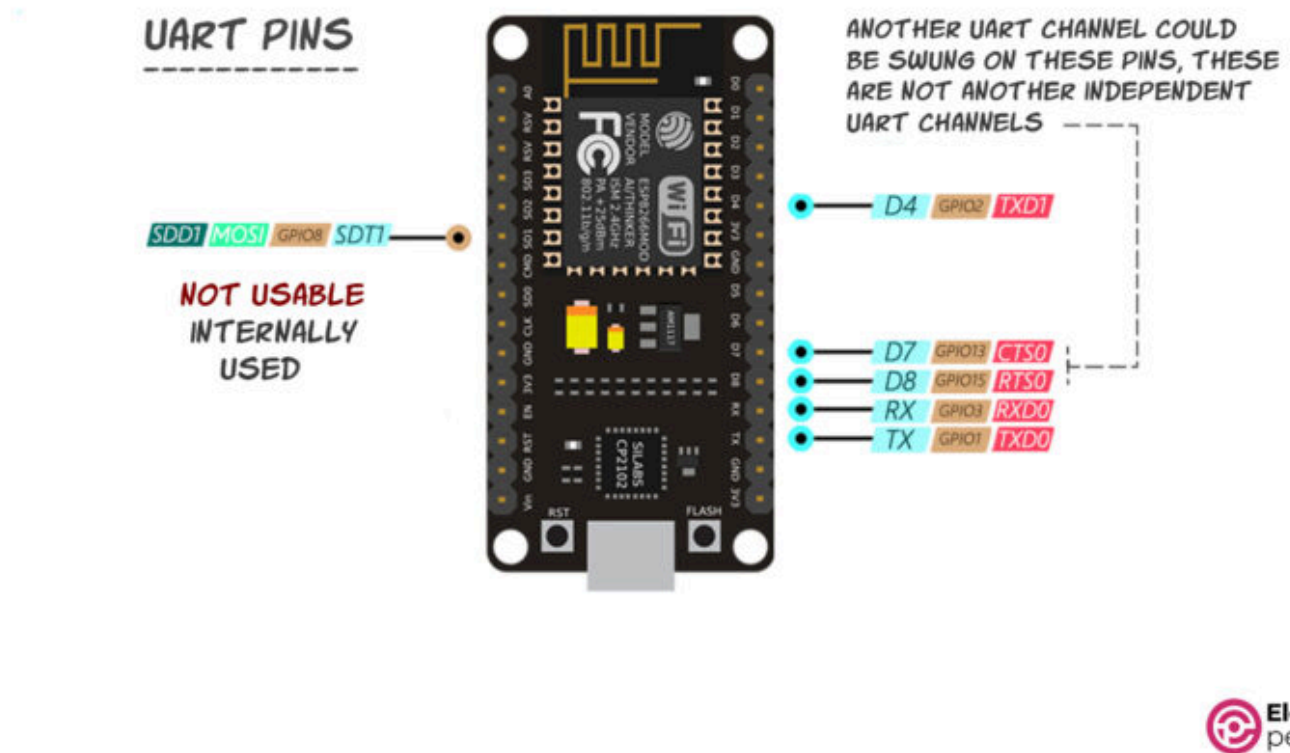
  Wire.requestFrom(8, 13); /* request & read data of size 13 from slave */
  while(Wire.available()){
    char c = Wire.read();
    Serial.print(c);
  }
}
```

```
18 | }
19 | Serial.println();
20 | delay(1000);
```

Code for Arduino Board

```
1 |
2 | #include <Wire.h>
3 |
4 | void setup() {
5 |     Wire.begin(8); /* join i2c bus with address 8 */
6 |     Wire.onReceive(receiveEvent); /* register receive event */
7 |     Wire.onRequest(requestEvent); /* register request event */
8 |     Serial.begin(9600); /* start serial for debug */
9 | }
10 |
11 | void loop() {
12 |     delay(100);
13 | }
14 |
15 | // function that executes whenever data is received from master
16 | void receiveEvent(int howMany) {
17 |     while (0 < Wire.available()) {
18 |         char c = Wire.read(); /* receive byte as a character */
19 |         Serial.print(c); /* print the character */
20 |     }
```

ESP8266 Pinout: Serial Protocol Pins (UART)



ESP8266-based NodeMCU has two UART communication ports: UART0 and UART1.

The RX/TX pins on the board, known as UART0, are functional.

- RX (GPIO3) RXD0 (Data Receive Pin)
- TX (GPIO1) TXD0 (Data Transmit Pin)

Only the data transmit pins of UART1 (TXD1/GPIO2/D4) are functional.

- D4 (GPIO2) TXD1 (Data Transmit Pin)

Tip

RXD2 and TXD2 pins are not separate for UART communications, and the NodeMCU ESP8266 Board only has two such UART channels. These pins are the only ones that we can use instead of the other UART channel pins. In other words, we can transfer the UART channel via these pins.

ESP8266 Pinout: PWM on ESP8266 Board with Arduino IDE

What is PWM?

By adjusting the on/off width of the output pulses, the “Pulse Width Modulation” technique controls the effective value of the output voltage. It enables us to control, for example, the speed of a DC motor or the brightness of an LED.

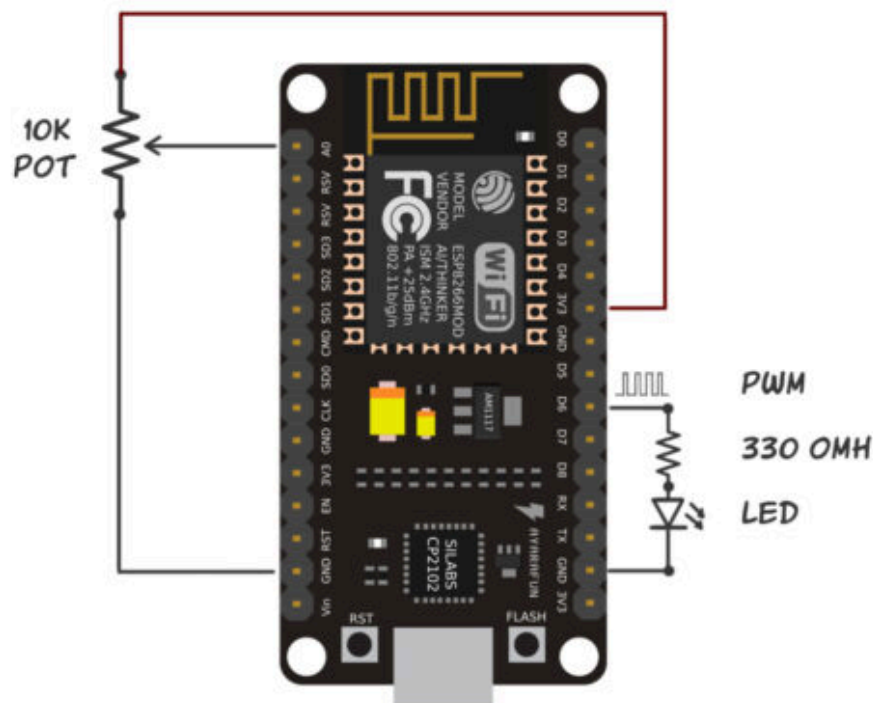
Software PWM is supported on all GPIO0 through GPIO15 pins by the ESP8266.

We can adjust the frequency range of the ESP8266 from 1Hz to 1KHz, generating a PWM signal with 10 bits of resolution.

Example:

Read the analog value from A0 and apply the corresponding PWM signal to D6 to increase/decrease an LED's light emission.

Schematic



Code

```
uint8_t LEDpin = D6;
```

```
3  /* By default PWM frequency is 1000Hz and we are using same
4   for this application hence no need to set */
5  void setup(){
6     Serial.begin(9600);
7     analogWrite(LEDpin, 512); /* set initial 50% duty cycle */
8  }
9
10 void loop(){
11     uint16_t dutycycle = analogRead(A0); /* read continuous POT and set PWM duty cycle a
12     if(dutycycle > 1023) dutycycle = 1023; /* limit dutycycle to 1023 if POT read cross i
13     Serial.print("Duty Cycle: "); Serial.println(dutycycle);
14     analogWrite(LEDpin, dutycycle);
15     delay(100);
16 }
```

ESP8266 Pinout: Interrupts

What is Interrupt or Interrupt Routine?

I'll explain it with an example. Say your building does not have a doorbell, and you live on the fifth floor. So, you must always be alert for knocks at the door. Now, what is the problem here?

First, you must check the door again and again. Secondly, if someone is at the door while you are doing something else, you have to wait until you're done and then go and check the door (well, in that case, the person at the door is already long gone). As you can see, this strategy is not working. What is the solution, then?

Buy and install a doorbell so you don't have to check the door all the time. After that, when the doorbell rings, you'll know someone is at the door and will stop working to open it (of course, if the thing you are doing is not very urgent). And when you open the door, you will get back to your task.

Interrupts work like a doorbell. For example, you specify an interrupt routine for the pin #n. If it happens, it must do a specific action. For instance, when GPIO4, which is the output of a sensor, goes to zero, an alarm system must be activated. If you have enabled the interrupt routine for the desired pin, as soon as such an event occurs (here, the GPIO4 going low), the program will stop, executing the defined routine and resuming where it left off.

Now, what if we use several interrupts and they happen all at the same time? (The doorbell and the phone may both ring simultaneously).

Here, first, you must take care of the task with greater priority, which has already been defined. In NODEMCU or any other microcontroller, several interrupt models are already prioritized.

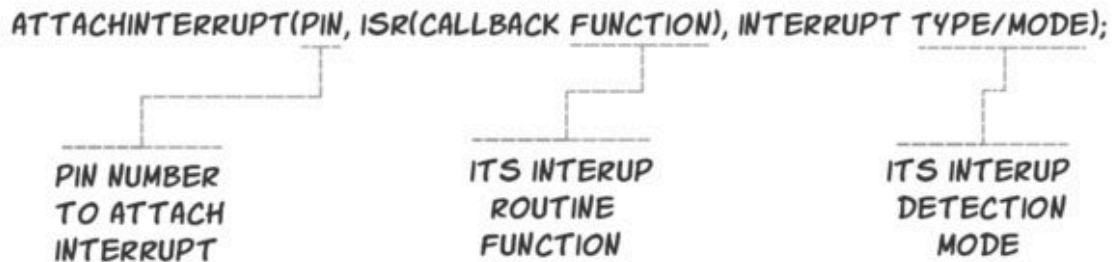
The I/O pin interrupt, for example, is an interrupt model. This feature is also included for the NodeMCU pinout and is pretty easy to use.

Interrupts can be used on D1 through D8 pins. D0 (GPIO16) does not have the interrupt feature (I wouldn't use this pin at all if I were you).

NodeMCU Interrupts in Arduino IDE

```
1 |  
2 | attachInterrupt(pin, ISR(callback function), interrupt type/mode);
```

The above syntax is used to apply the interrupt routine to a specific pin. The description of its different parts is given in the image below.



```

1 |
2 | detachInterrupt(pin)

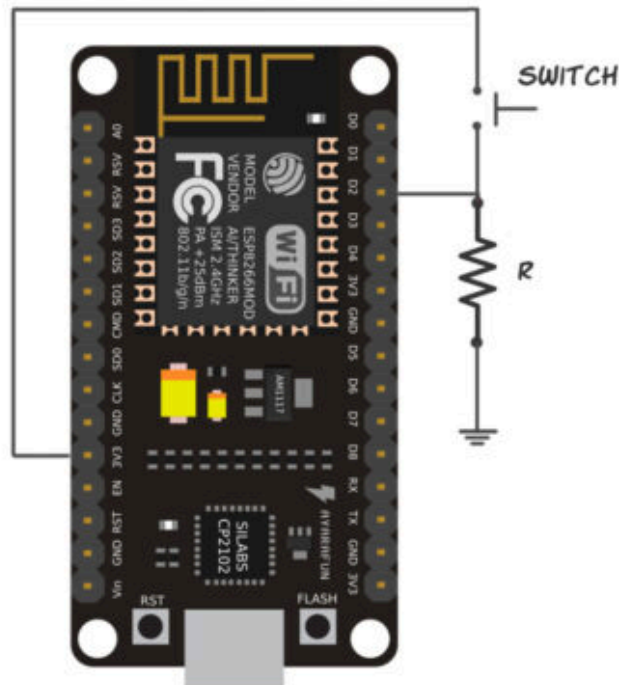
```

The above syntax is used to deactivate a pin interrupt.

Example:

We are going to write a program that detects interrupts on the rising edge of a pin—when the desired pin goes from LOW to HIGH—and display the time it happens (in milliseconds) in the serial window.

Interrupt Circuit Schematic of NodeMCU ESP8266 Board



NodeMCU Interrupt Code using Arduino IDE

```

1
2  uint8_t GPIO_Pin = D2;
3
4  void setup() {
5      Serial.begin(9600);
6      attachInterrupt(digitalPinToInterrupt(GPIO_Pin), IntCallback, RISING);
7  }
8
9  void loop() {
10 }
11
12 void IntCallback(){
13     Serial.print("Stamp(ms): ");
14     Serial.println(millis());
15 }

```

ESP8266 Pinout: Pins Involved at Boot

- **GPIO16 /D0** pin is high at BOOT
- **GPIO0 /D3** boot failure if pulled LOW
- **GPIO2 /D4** pin is high at BOOT, boot failure if pulled LOW
- **GPIO15 /D8** boot failure if pulled HIGH
- **GPIO3 /RX** pin is high at BOOT
- **GPIO1 /TX** pin is high at BOOT, boot failure if pulled LOW
- **GPIO10 /SD3** pin is high at BOOT
- **GPIO9 /SD2** pin is high at BOOT

ESP8266 Pinout: Control Pins

EN Pin (Enable)

The ESP chip is active when the EN pin goes HIGH. When LOW, the ESP chip is in the minimum power consumption mode. In some models, the pin's name might be CH_PD.

RST pin (Reset)

The reset pin of ESP8266, or RST, is HIGH by default. When it goes LOW, it instantly resets the ESP8266. It works like pressing the RESET button on the board.

FLASH/ D3/GPIO0 Pin

If you hold this pin low and pull the EN pin to low, the board will go into the Flashing Mode.

If you hold this pin low and pull the RESET button to low, the board will go into the Uploading Mode.

This pin serves the same purpose as the FLASH button on the board.

WAKE / GPIO16 / D0 Pin

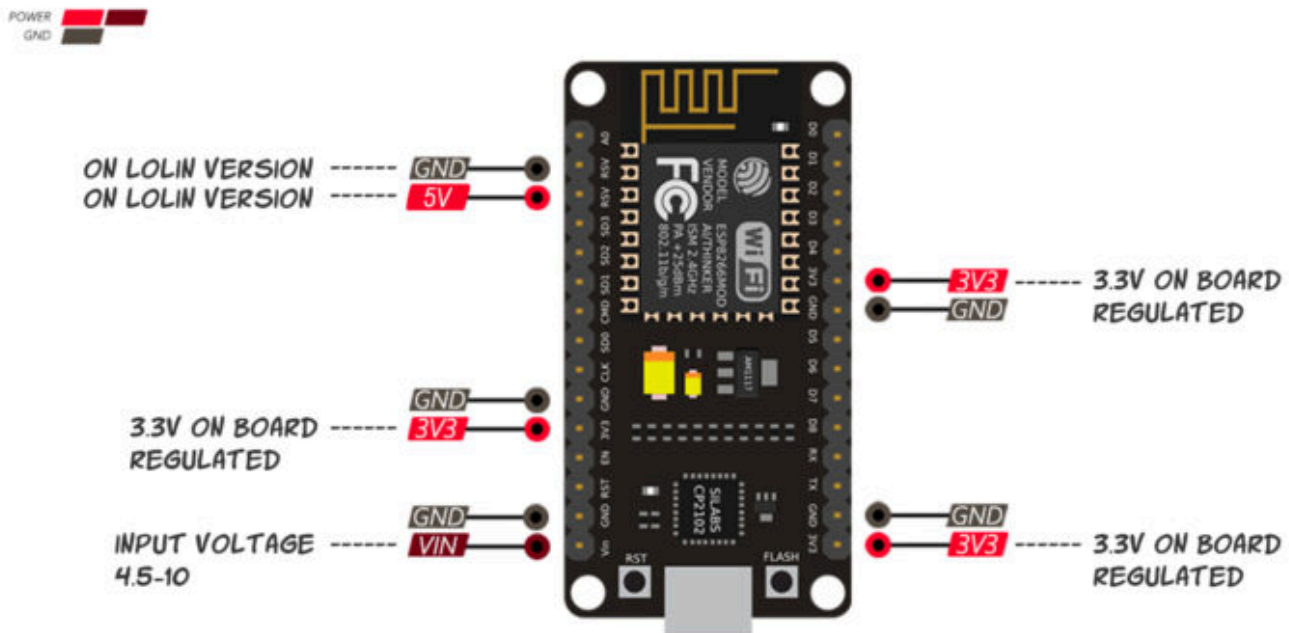
What does “Deep Sleep Mode” mean? The RTC (real-time counter) is the only active unit in this mode, resulting in the lowest energy consumption.

The WAKE pin is used to wake the ESP8266 from Deep Sleep Mode. To use this feature, you need to connect this pin to the reset pin. Now, what does it mean? When the board is on, the RST pin is HIGH. And the board is reset as the RST pin goes LOW. When we configure a deep sleep timer with a wake-up time, GPIO16 goes LOW as the specified time comes to an end (this is the pin’s wake feature). With GPIO16 connected to RST, this pin also goes LOW at the same moment and triggers a reset, which forces the board to exit from the Deep Sleep Mode.

This pin is HIGH at boot and goes down to 1V.

ESP8266 Pinout: Power Line Pins

The power line pins are shown in the figure below.



- **Vin**: This is the only input voltage pin of the circuit (except for the USB port's input voltage, of course). The input voltage of the NodeMCU Board ranges from 4.5 to 10V.
- **GND**: ground.
- **3v3**: These are the output voltage pins of the board's regulator. The maximum output current is 600 mA; try not to get even close to this number. In order to set up other circuits that consume more than a few tens of mA, instead of using the microcontroller board as a source, it's better to have an external source (in that case, remember to connect all the GNDs to one another).
- **5V***: The Pin 3 on LoLin version boards serves as the 5V output voltage pin. This pin in the Amica version is the RSV pin and is not connected anywhere.

Programming NodeMCU ESP8266 in Arduino IDE

Perhaps one of the factors contributing to the popularity of these boards is the ability to program NodeMCU boards using the Arduino software. Working with ESP8266 in Arduino IDE requires just a few simple steps.

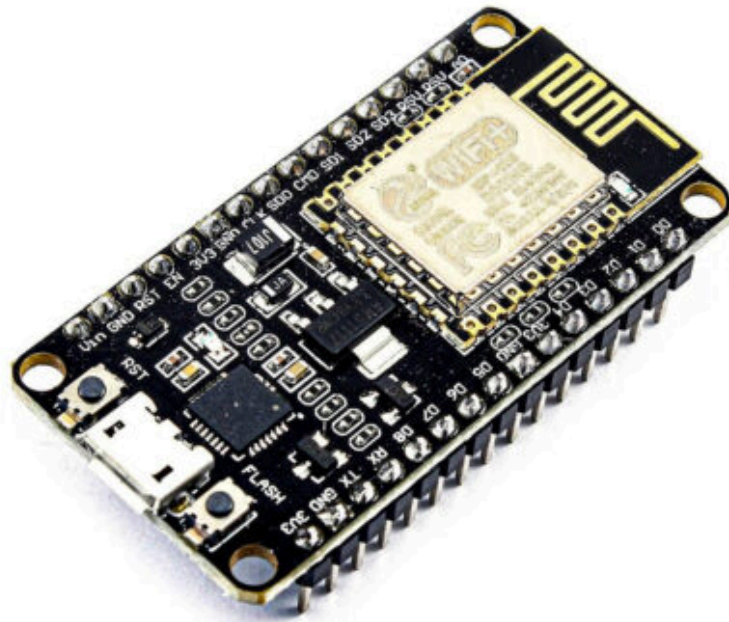
Hardware Components

NODEMCU ESP8266

x

1





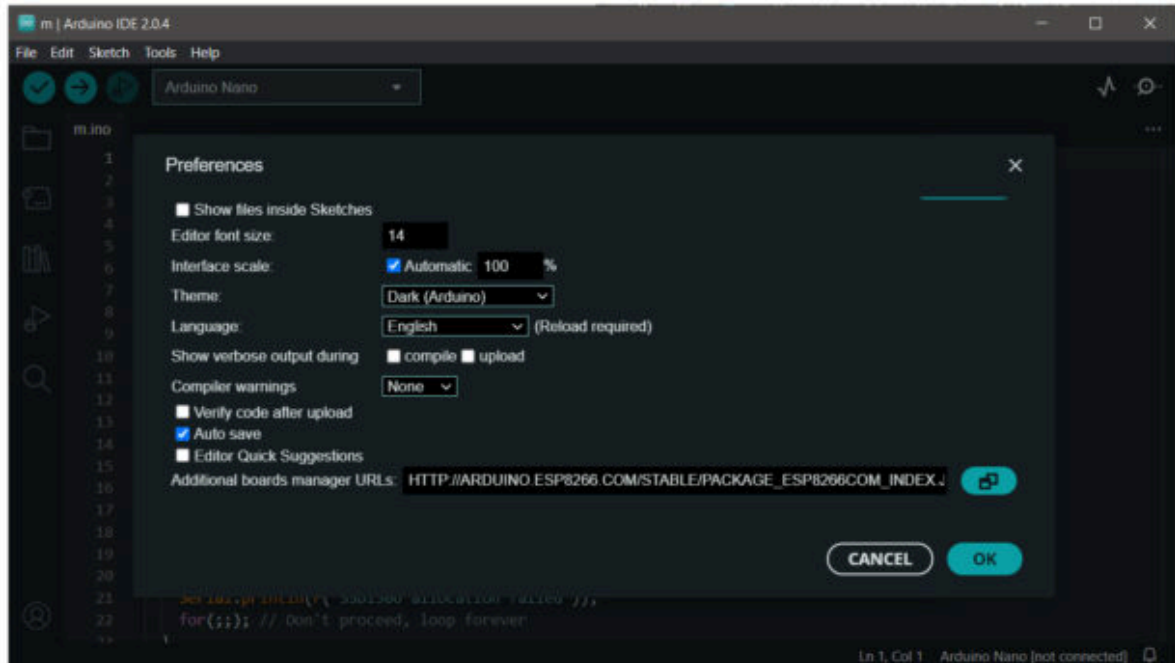
Software

Arduino IDE



Configuring Arduino IDE Software for NodeMCU

- 1- First, download the latest version of Arduino IDE software from the link above.
- 2- After installing the Arduino IDE Software, run the program and open the following window from File>preferences.



3- Copy the link below and paste it into the box as shown in the figure. Then press OK.

[HTTP://ARDUINO.ESP8266.COM/STABLE/PACKAGE_ESP8266COM_INDEX.JSON](http://arduino.esp8266.com/stable/package_esp8266com_index.json)

4- Now go to Tools>Board>Board Manager. In this window, type in “ESP8266” in the box, find “ESP8266 by ESP8266 community” as shown in the image, and then click the install option.



5- Next, select the desired ESP model from Tools > Board.

Now you can write codes for both ESP8266 modules and NodeMCU boards based on ESP8266 in the Arduino IDE Software.

Wrapping Up

The development boards known as NodeMCU, which are based on the ESP8266 or ESP32 microcontrollers, have made it much easier to work with these microcontrollers by providing additional features. These boards are a good option for all kinds of IoT applications because of their Wi-Fi connection—a well-known feature of these microcontrollers—simple process setup and use, and cheap pricing compared to similar boards.

As for the GPIO pins on the NodeMCU ESP8266 board, it should be noted that only the pin A0 is available as an analog input from the pins on the left side of the board (when you hold the board in a way that the USB port is at the bottom and the antenna is at the top). GPIO6 to GPIO11, which are on the same side, are not intended for external use. The most suitable options for digital input/output are D1 (GPIO5) and D2 (GPIO4) pins. These pins also play the role of I2C Protocol pins. Pins D5, D6, D7, and D8 are SPI

communication pins. And for Serial Communications, RX and TX pins are employed. Perhaps one of the drawbacks of NodeMCU boards is the small number of GPIO pins available on them. For projects that require more inputs/outputs, we can use I/O Extension Interfaces.

For programming and running ESP boards, you can use the Arduino IDE. All you have to do is add the required board in the board manager section of the software.

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐ Save my name, email, and website in this browser for the next time I comment.

☐

I'm not a robot

reCAPTCHA

POST COMMENT



- +86 139 234 941 92
- info@electropeak.com
- Shenzhen, China

COMPANY

- About us
- Contact us
- My Account

MY ACCOUNT

- My Wishlist
- My Account
- Order History

CUSTOMER SERVICE

FAQ

[Return Policy](#)

[Payment & Shipping](#)

ElectroPeak Inc. © 2019. All Rights Reserved

WORKING DAYS/HOURS

Mon – Fri / 9:00-18:00 GMT+8



© All rights reserved

